# Refleks Testi: Oyun Tasarımı ve İşleyişi, Oyun Geliştiricinin Katkıları

Ayşe Ayhan
*Eskişehir Osmangazi Üniversitesi*
Bilgisayar Mühendisliği Eskişehir,
Türkiye
152120201058

***Abstract*— Refleks Testi, oyuncunun dikkat ve refleks yeteneğini ölçmeyi amaçlayan, farklı zorluk seviyelerine sahip bir aksiyon oyunudur.**

**Bu sistem, oyun içinde şekillere özgü puanların kolayca ölçülmesini hedefler. Oyun; Easy, Mid ve Hard olmak üzere üç seviyeye sahiptir. Her seviyede farklı aktör türleri (küre, küp ve koni) ve puanlama sistemleri sunulmaktadır.**

**Bu raporda, Refleks Testi'nin temel tasarımı, mekanikleri ve teknik altyapısı sunulmaktadır.**
***Index Terms*—Oyun Tasarımı, Refleks Testi, Blueprint Yönetimi, Unreal Engine, UI Tasarımı, Puanlama Sistemi**

## I. GİRİŞ

Günümüzde video oyunları, eğlence sektörü içerisinde en hızlı büyüyen alanlardan biridir. Teknolojinin ilerlemesiyle birlikte oyun geliştirme süreçleri daha yenilikçi yaklaşımlar sunmakta ve oyunculara daha sürükleyici deneyimler sağlamaktadır. **Refleks Testi**, oyuncuların dikkat ve refleks yeteneklerini ölçmeyi amaçlayan, farklı zorluk seviyelerine sahip bir aksiyon oyunu olarak bu alana katkı sunmayı hedeflemektedir. Bu rapor, **sadece geliştiricinin katkılarını** yansıtmakta, oyuna dair tüm detayları içermemektedir.

## II. OYUN SEVİYELERİNDEKİ PUANLAMALAR

**Refleks Testi**'nde Her bir şekil türü için bir **Enum** değeri (**E_ShapeType**) tanımlanmıştır. Bu Enum, farklı şekillerin kolayca ayırt edilmesini sağlarken, her birine özel puanlama yapılmasına olanak tanır. AddScore fonksiyonu ile şekillerin puanı atanır.

Aşağıdaki puanlama sistemi kullanılmıştır:

**Küre:** Kolay seviyede en düşük puan değeri atanır. Hedeflerin büyük ve hareketlerinin yavaş olması sebebiyle başlangıç seviyesi için uygundur.

**Küp:** Orta seviyede, kürelere kıyasla daha yüksek puan değeri taşır. Daha zor hedef alma ve hızlı reaksiyon gerektirir.

**Koni:** Zor seviyede en yüksek puan değeri atanır. Küçük boyutu ve karmaşık hareketleri ile refleks testinin en zorlu bölümünü oluşturur.

- `Küre (BP_ShapeBall)`: 50 puan
- `Koni (BP_Cone)`: 100 puan
- `Küp (BP_Cube)`: 80 puan

### *A. ShapeBall BluePrint:*

- **Küre:** Ekranda **1-3 saniye** arasında rastgele bir süreyle görünür. Bu, oyuncunun dikkat süresini ölçmek ve hızlı reaksiyon göstermesini sağlamak için tasarlanmıştır.
- **Puanlama:** Küre vurulduğunda oyuncuya **50 puan** kazandırır. Bu, başlangıç seviyesi bir hedef için uygun bir ödül değeridir.
- **Görünüm:** Kürenin materyali ve görünümü, **Fig. 2.**'de gösterildiği gibi tasarlanmıştır. Bu tasarım, görsel açıdan dikkat çekici ve kolay algılanabilir olmasını sağlamaktadır



Fig. 2. Kolay Modda ekranda oluşan küre

### *B.* BP_CubeShape *BluePrint:*

- **Küp:** Ekranda **3-5 saniye** arasında bir süreyle görünür. Bu, oyuncunun hedefi takip etme ve doğru zamanda vurma yeteneğini test etmek için planlanmıştır.
- **Puanlama:** Küp vurulduğunda oyuncuya **80 puan** kazandırır. Bu, kürenin üzerindeki zorluk seviyesi için uygun bir ödül değeridir.

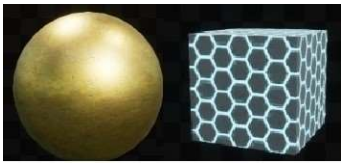


Fig. 3. Orta Modda ekranda olus¸an Ku¨re ve Ku¨p

- **Küre Görünümü ve Materyali:** Kürenin görünüm ve materyali, **Fig. 3.**'te gösterildiği gibi seçilmiştir. Tasarım, kullanıcıların kolayca fark edebileceği, görsel olarak diğer şekil materyallerinden de farklılaşarak hoş bir deneyim sunacak şekilde hazırlanmıştır.

*C.* BP_ConeShape *BluePrint:*

**Koni:** Ekranda **2-4 saniye** arasında bir süreyle görünür. Bu, oyuncunun daha hızlı refleks göstermesini ve karmaşık hedeflere odaklanmasını sağlamak için tasarlanmıştır.

**Puanlama:** Koni vurulduğunda oyuncuya **100 puan** kazandırır. Bu, en yüksek zorluk seviyesi hedefi için uygun bir ödül değeridir.

**Görünüm ve Materyal:** Koninin görünüm ve materyali, **Fig. 4.**'te gösterildiği gibi seçilmiştir. Materyal tipi ve rengi, yüksek zorluk seviyesine uygun şekilde seçilmiştir.

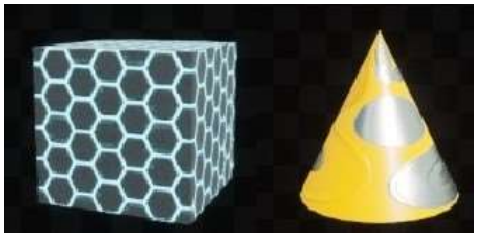


Fig. 4. Zor Modda ekranda oluşan Küp ve Koni

## III. AKTÖRLER VE SPAWNER MEKANİĞİ

*A. Aktörler*

**Yerleştirme:** Aktörler, **ShapeSpawner Blueprinti** aracılığıyla ekrana rastgele yerleştirilir. Bu mekanizma, her bir aktörün oyun alanında farklı bölgelerde görünmesini sağlar ve oyuncunun dikkatini dinamik bir şekilde yönlendirir.

**Görünme Süresi:** Aktörlerin ekranda kalma süresi, oyuncunun baktığı kamera açısı ile uyumlu olacak şekilde, ekran çerçevesi içinde rastgele olarak ayarlanır. Bu, aktörlerin görünürlük süresini optimize ederek oyuncunun deneyimini daha etkileyici hale getirir.

*Bu işlem için **Get Camera Location**, **Random Değerler** ile **Make Vector**, **Multiply** ve **Make Transform** işlemleri kullanılmıştır. Bu adımlar, aktörlerin ekranın farklı noktalarına rastgele yerleştirilmesini ve oyuncunun bakış açısına göre dinamik bir görünürlük sağlanmasını mümkün kılar.*

*B. Hit Event ve Aktör Yok Edilmesi*

**On Component Hit Event:**
Oyuncu, aktörleri silah ile vurduğunda, **On Component Hit** eventi tetiklenir. Bu event, oyuncunun hedefe isabet etmesini algılar ve ardından gerekli işlemleri başlatır.

**Aktörün Yok Edilmesi (Destroy Actor):**
Bu event, vurulan aktörün **Destroy Actor** komutuyla yok edilmesini sağlar. Aktör ekrandan silinir ve oyuncuya hedefi vurmanın ödülü verilir.

**Puan Ekleme:**
Aktör yok edildikten sonra, oyuncuya ilgili aktörün puanı eklenir. Örneğin, küre için 50 puan, küp için 80 puan ve koni için 100 puan kazandırılır. Bu işlem, oyuncunun doğru hedeflere odaklanmasını ve başarısını ölçer.

Şekillerin oluşturulması, tasarımı, **şekil aktör BluePrintleri**, **ShapeSpawner BluePrint** tasarımı, aktör component hit, puanlama, **AddScore fonksiyonu**, konum ve süre rastgeleliği görevlerini geliştirici gerçekleştirmiştir.

## IV. REFLEXGAMEMODE VE EK ÖZELLİKLER

*Fonksiyonlar*

- *AddScore:*
  Oyuncunun bir aktörü vurması durumunda tetiklenen bu fonksiyon, ReflexGameInstance içerisinde bulunur. Fonksiyon, vurulan aktörün türüne göre belirlenmiş puanı her bir şekil için Score değişkenine ekler.
    - Küre vurulduğunda 50 puan, Küp vurulduğunda 80 puan, ve Koni vurulduğunda 100 puan eklenir.

  Bu fonksiyon, oyuncunun toplam puanını yönetir ve günceller. ReflexGameInstance, oyuncunun oyun içindeki genel durumu ve ilerlemesini takip eden merkezi bir sınıf olarak AddScore fonksiyonunu barındırır. Bu sayede oyuncu puanları tüm oyun boyunca saklanabilir ve erişilebilir olur. Geliştirici burada **GameInstance** oluşturulması, **ReflexGameMode** içerisinde, bahsedilen işlemler ve **AddScore** fonksiyonunun oluşturulmasında görev almıştır.

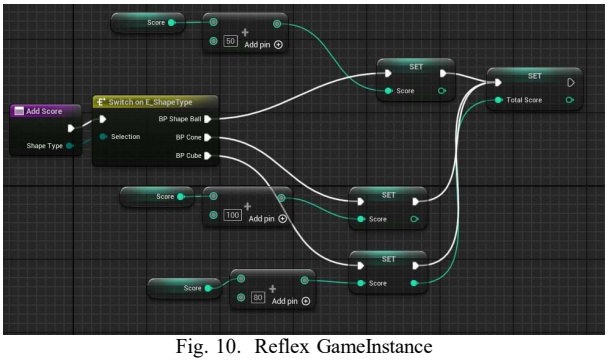



Fig. 10. Reflex GameInstance



Fig. 11. Oyunun Oynanışı

## V. SONUÇ VE TARTIŞMA

**Refleks Testi**, oyuncuların dikkat ve refleks becerilerini ölçen etkili bir oyun yapısı sunmaktadır. Oyun, **Unreal Engine** ve **Blueprint tabanlı mimari** kullanılarak geliştirilmiş olup, bu teknolojiler oyun tasarımı ve geliştirme süreçlerinde güçlü bir referans niteliği taşımaktadır.

**Sonuçlar:**
Refleks Testi, oyunculara farklı zorluk seviyelerinde dinamik ve ilgi çekici bir deneyim sunmayı hedeflemektedir. Oyunun, oyuncu becerilerini ölçmek için geliştirilmiş **aktörler**, **puanlama sistemi** ve **spawner mekanizmaları**, oyun tasarımının verimli bir şekilde işlediğini göstermektedir. Ayrıca, **ReflexGameMode** ve **ReflexGameInstance** gibi temel sistemler, oyuncunun oyun içindeki ilerlemesini düzgün bir şekilde izler ve yönetir.

## REFERENCES


[1] Unreal Engine Documentation, ”Blueprint Visual Scripting,” Epic Games, [Online]. Available: https://docs.unrealengine.com/en-US/Blueprints/. [Accessed: Dec. 22, 2024].

[2] Unreal Engine Documentation, ”Actors and Components,” Epic Games, [Online]. Available: https://docs.unrealengine.com/en-US/Actors/. [Accessed: Dec. 20, 2024].

[3] Unreal Engine Documentation, ”UMG UI Designer,” Epic Games, [Online]. Available: https://docs.unrealengine.com/en-US/UMG/. [Accessed: Dec. 19, 2024].